Study Sprint 3 Quest Studio

Software Design Patterns / Final project report

Introduction

Study Sprint is a Java 17 and Swing planner for students managing several courses. Quest Studio expands the planner with a course map, CSV task import and undo for completion changes. The app retains schedules, timed study, the daily mission and report export.

Overview shows level, completed tasks, timed minutes and active streak. The weekly chart switches between XP and timed focus minutes.

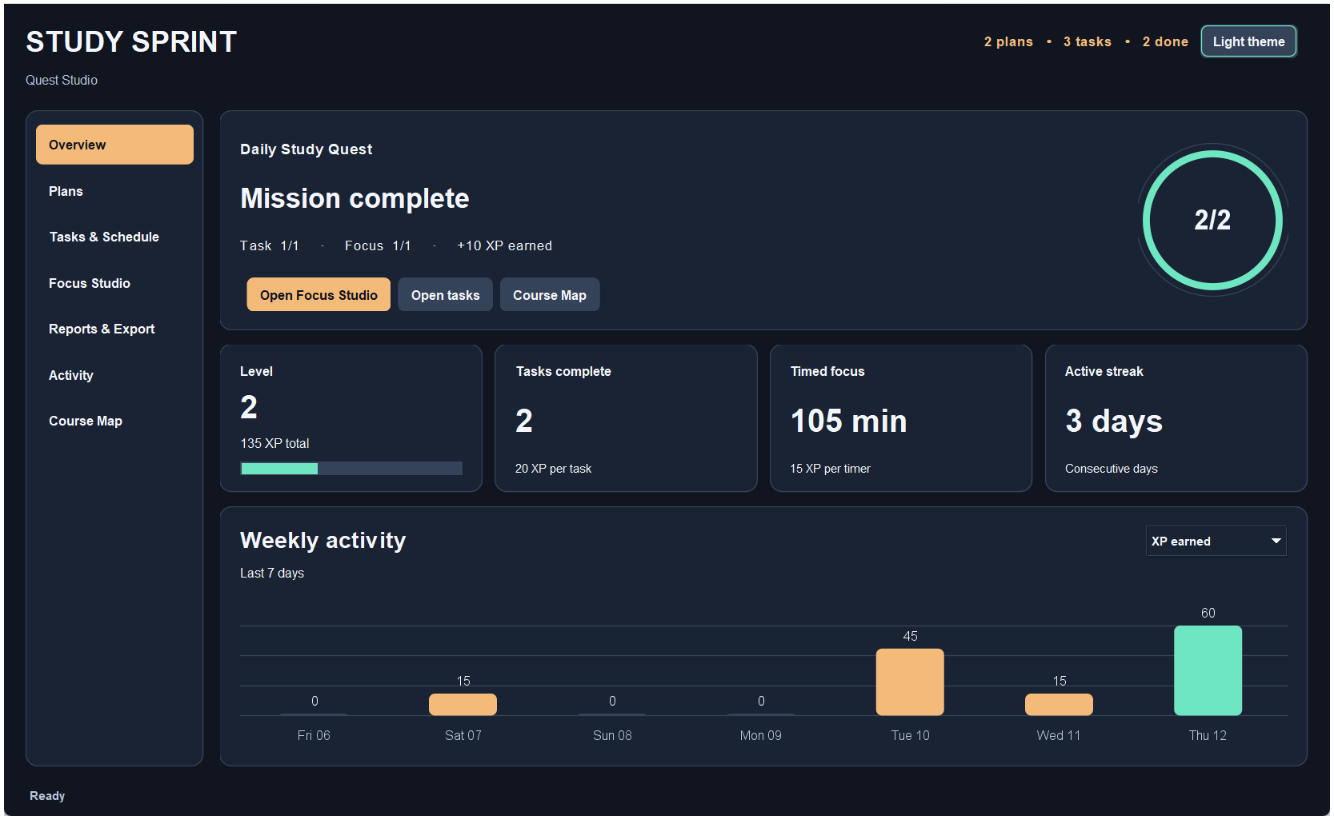


Figure 1. Dark Overview with illustrative local history. The mission is complete and the active streak is three days.

Run the application

Install JDK 17. From the extracted project folder run .\scripts\run.bat on Windows or bash scripts/run.sh on Linux or macOS. Two sample plans, three tasks and one manual session appear on first launch.

User Scenarios

1. Create a study plan with a title and goal. Edit the goal as coursework changes.

2. Add two tasks with distinct estimates and priorities. Compare Priority first and Shortest first in the schedule preview.

3. Complete a task and check Activity for the notification and Overview for the first mission step.

4. Select a task in Focus Studio. Start a one-minute demonstration timer, pause and resume, then select Finish & save at zero. Overview marks the mission complete.

5. Select report sections and export TXT or CSV. Restart with the same data file to verify the saved task, timer and progress.

6. Preview and import the sample task CSV. Inspect the new tasks in Course Map, then use Undo and Redo to demonstrate a reversible completion change.

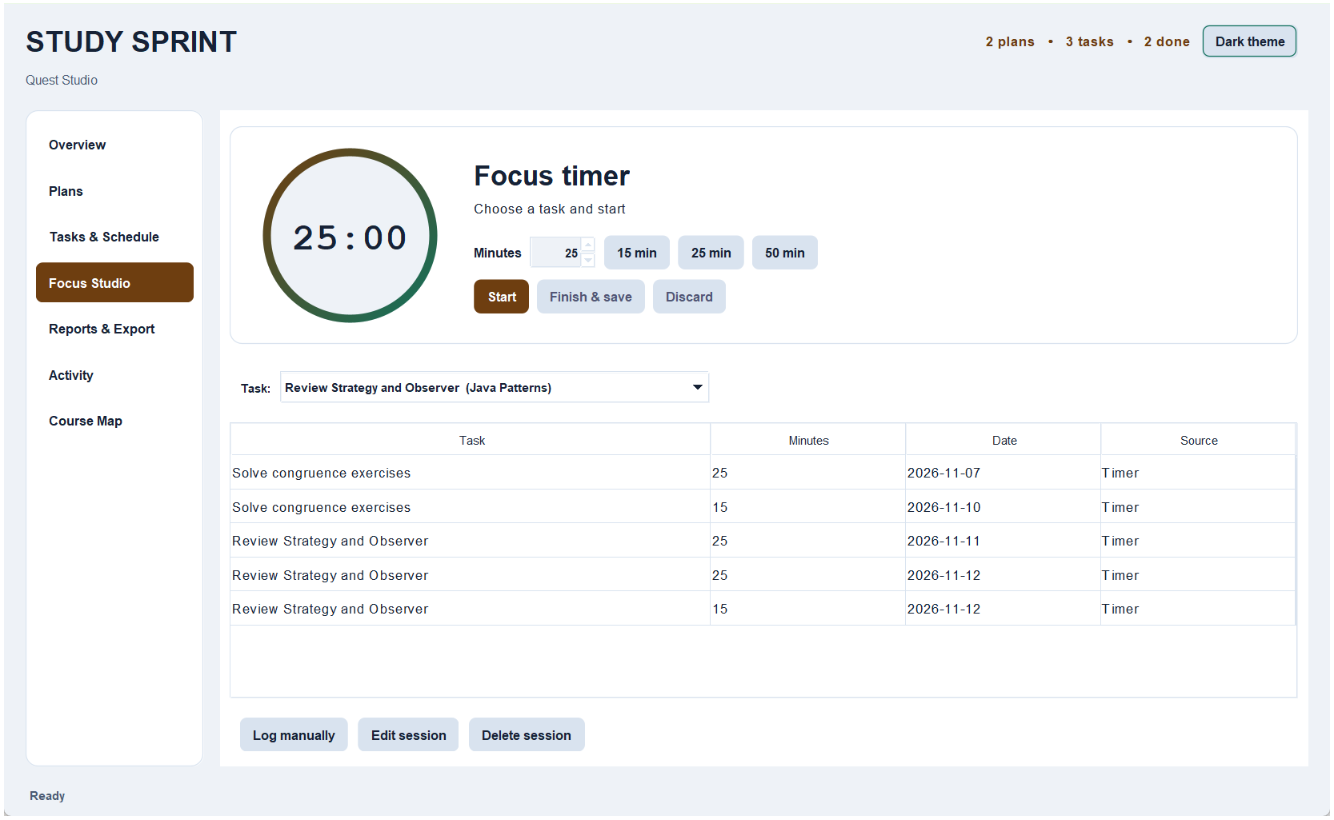


Figure 2. Light Focus Studio screen with the timer dial, 15, 25 and 50 minute presets, and session history.

Architecture and Core Patterns

Swing handles interaction and rendering. StudyPlannerFacade coordinates validation, reports, scheduling and persistent changes. The repository writes a complete snapshot before an event updates the UI. StudyPlan owns tasks, and each FocusSession refers to one StudyTask.

Pattern	Problem	Participants	Demonstration	Student
Builder	Construct validated tasks with several fields.	StudyTask.Builder, StudyTask	Add or edit a task.	_____
Factory Method	Select a formatter through a common export workflow.	ExportService, TextExportService, CsvExportService, ReportFormatter	Export TXT, then CSV.	_____
Decorator	Compose optional report sections.	SummaryComponent, BaseSummary, SummaryDecorator, ProgressSummaryDecorator, FocusSummaryDecorator	Toggle progress and focus in the preview.	_____
Facade	Keep UI actions independent of storage and reports.	PlannerFrame, StudyPlannerFacade, StudyRepository	Save a completed focus timer.	_____
Strategy	Swap scheduling rules at runtime.	SchedulingStrategy, PriorityStrategy, ShortestTaskStrategy	Switch between urgent and short tasks.	_____
Observer	Refresh views after a committed change.	StudyEventHub, StudyEventListener, StudyChangeEvent, PlannerFrame	Complete a task and check Activity and Overview.	_____

Enter each responsible student’s real name before submission.

Persistence

The UTF-8 Properties file stores task completion dates and timed session provenance in the same snapshot as plans and tasks. Version 1 data remains readable. Older completed tasks retain 20 XP each, with no date inferred for the daily mission or chart.

Additional Patterns in Quest Studio

Quest Studio adds four patterns with visible behavior. Adapter connects CSV to the existing task builder, Composite shares workload calculations, Command records reversible actions, and State defines timer transitions.

Pattern	Problem	Participants	Demonstration	Student
Adapter	Read two CSV layouts as task data.	TaskImportSource, CsvTaskImportAdapter, CsvTableReader, TaskDraft	Preview and import sample-data/tasks-import.csv.	_____
Composite	Calculate progress across levels of the course map.	WorkloadNode, CourseMap.Branch, CourseMap.TaskNode, CourseMapPanel	Select workspace, plan and task to compare totals.	_____
Command	Reverse task completion while keeping other edits.	StudyCommand, ToggleCompletionCommand, CommandHistory, StudyPlannerFacade	Complete a task, then Undo and Redo.	_____
State	Define timer behavior for each phase.	FocusTimer.TimerState, Idle, Running, Paused, Finished	Start, pause, resume and finish at zero.	_____

Import and undo behavior

CSV import previews the task rows and saves them as one batch. If any task fails validation, the batch is rejected. Import can also read task rows from a Study Sprint report. New tasks start unfinished.

Undo and Redo apply to task completion, including the date used by Study Quest. The saved task state survives restart. Undo history lasts for the current session and is cleared when a task or plan is deleted.

Course Map and Workload

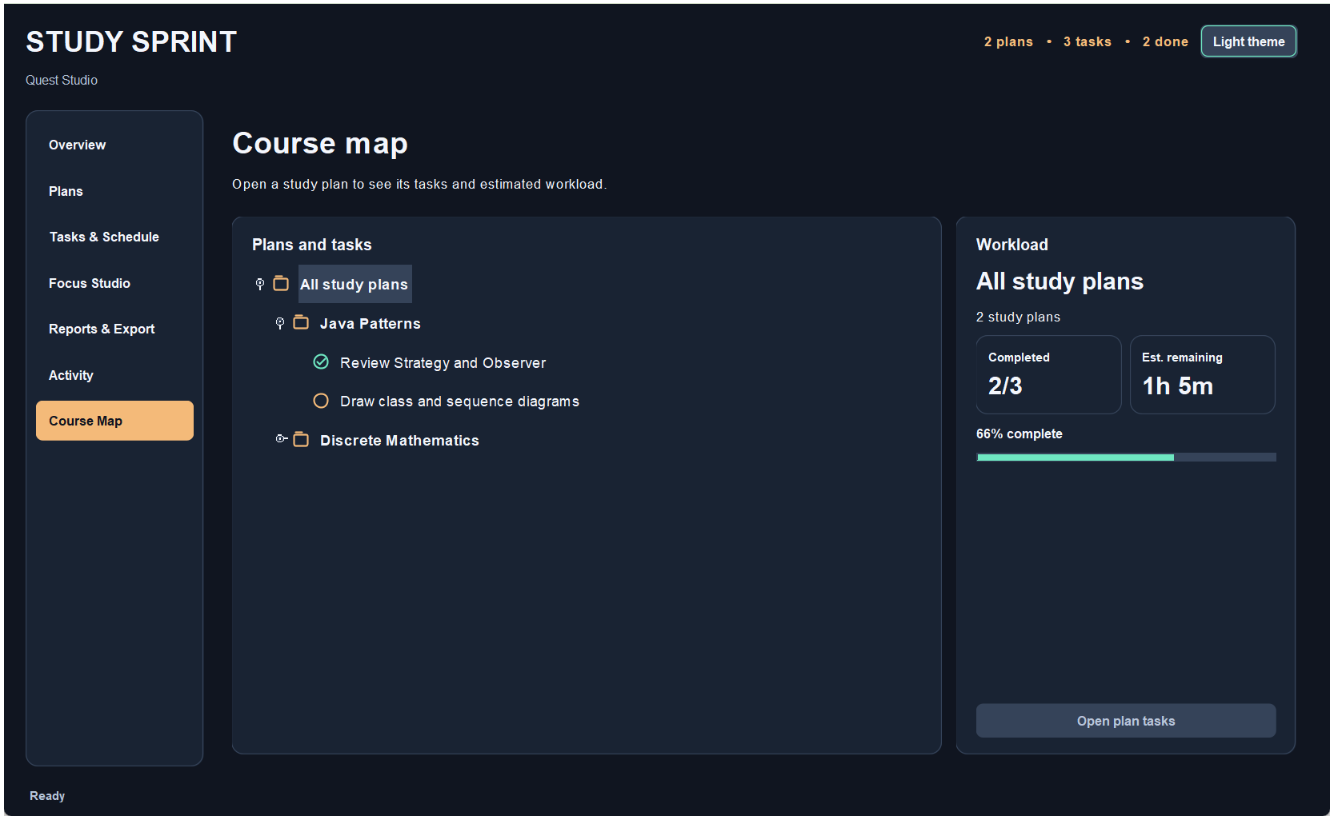


Figure 3. Course Map with the workspace selected. The workload card aggregates two plans and three tasks.

The course map shows the estimated work left for a whole workspace, a plan or a task. Completing a task removes its estimate from the remaining workload. Select a plan or task and choose Open plan tasks to continue working.

UML Class Diagram

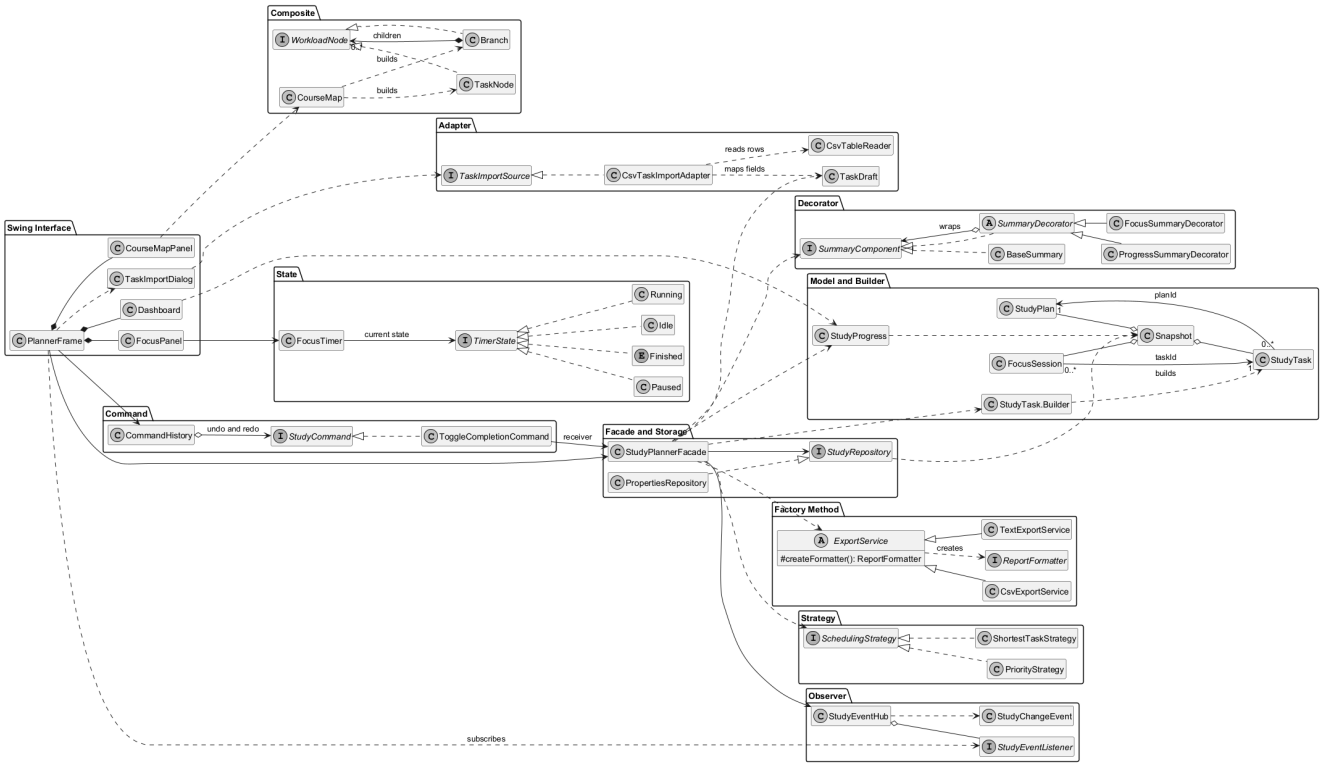


Figure 4. Domain entities, UI, repository and the participants in the ten patterns. Open the separate PNG for full-size labels.

Responsibilities

FocusPanel owns the running countdown. FocusTimer measures active time with a monotonic clock. The facade saves the completed timer and calculates a StudyProgress view of the current snapshot. Dashboard presents this view and does not write persistent data.

PriorityStrategy and ShortestTaskStrategy order the same unfinished tasks. Summary decorators wrap a shared component and can be combined. The export service subclasses supply different formatter implementations.

UML Sequence Diagram

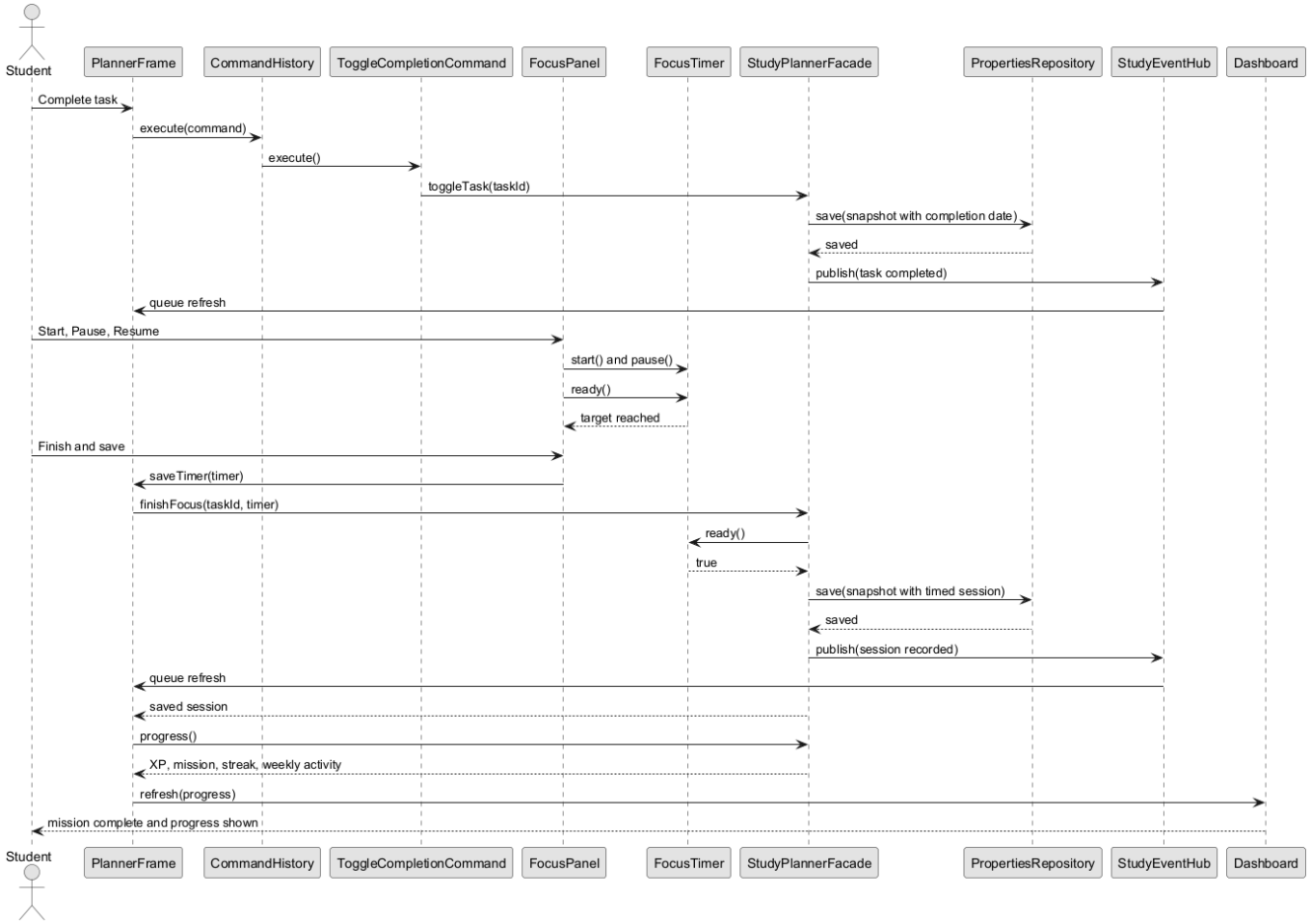


Figure 5. Completing a task and saving a timer satisfy the daily mission.

Progress rules

A currently completed task contributes 20 XP, a saved timer contributes 15 XP, and a qualifying mission adds 10 XP per day. Level 1 begins at zero XP and each additional 100 XP increases the level. Manual logs remain available for reports.

Reopening a task reverses its contribution. Completing it again restores the original completion date. Editing the minutes or date of a timed record converts it to a manual record. An active streak ends today, or yesterday if there is no activity today.

Java Implementation Excerpts

Builder and Facade

The facade constructs validated tasks through the builder and accepts a timer only after its full active interval.

```
StudyTask task = StudyTask.builder(UUID.randomUUID().toString(), planId).title(title)
    .estimateMinutes(minutes).priority(priority).dueDate(due).build();

if (!timer.ready()) throw new IllegalArgumentException("Finish the countdown first");
return saveSession(new FocusSession(timer.id(), taskId,
    timer.targetMinutes(), LocalDate.now(clock), true));
```

Strategy and Decorator

The chosen schedule strategy orders tasks. Each report option wraps the same summary component.

```
return List.copyOf(Objects.requireNonNull(strategy).order(tasks(planId)));

SummaryComponent summary = new BaseSummary(plan);
if (progress) summary = new ProgressSummaryDecorator(summary, tasks);
if (focus) summary = new FocusSummaryDecorator(summary, sessions);
```

Factory Method and Observer

The export creator delegates formatter creation to subclasses. The facade publishes the change after the repository saves it.

```
protected abstract ReportFormatter createFormatter();

repository.save(next);
state = next;
events.publish(message);
```

Composite and State

```
for (WorkloadNode child : children) {
    total = total.plus(child.progress());
}

public void start() { state = state.start(nanoTime.getAsLong()); }
public void pause() { state = state.pause(nanoTime.getAsLong()); }
```

Excerpts from StudyPlannerFacade, ExportService, CourseMap and FocusTimer. Some long lines are wrapped for print.

Testing and Individual Defense

Java 17 automated run: 23 passed, 0 failed. Reproduce with .\scripts\test.bat on Windows or bash scripts/test.sh on Linux or macOS. The project contains the full expected and actual results in docs/test-scenarios.csv and the execution log in docs/test-results.txt.

Tests	Coverage	Observed result
T01–T03	Seed, create, edit, reload	Sample and modified data loaded correctly.
T04–T06	Strategy, Observer, Decorator	Order, notification and preview changed.
T07–T08	Session CRUD and exports	Manual journal and both export formats worked.
T09–T13	Invalid inputs and cascade	Invalid requests rejected and related records removed.
T14–T18	Timer, quest, XP, dates, migration	Pause excluded, no duplicate XP, old files loaded.
T19–T23	Command, CSV import, course map, State	Undo restored saved state, failed imports were atomic, totals matched, timer stayed finished.

Individual contributions

Team members and actual code contributions _____

GitHub or GitLab repository URL _____

Each student should demonstrate the patterns they implemented, explain their participants and trade-offs, and independently make a small change during the defense.

Conclusion and future work

Quest Studio connects imported tasks, workload, timed study and saved progress in one desktop workflow. Future work could add configurable mission targets and weekly history navigation.

AI disclosure. Codex assisted with the implementation update, report and verification. Record each team member’s actual work and disclose tool use under course policy.